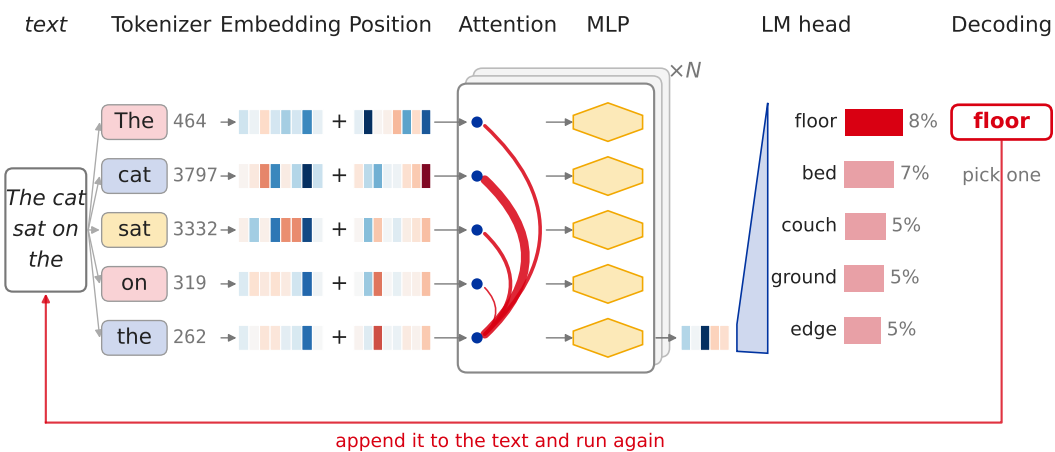
Post-training

From a base model to an assistant

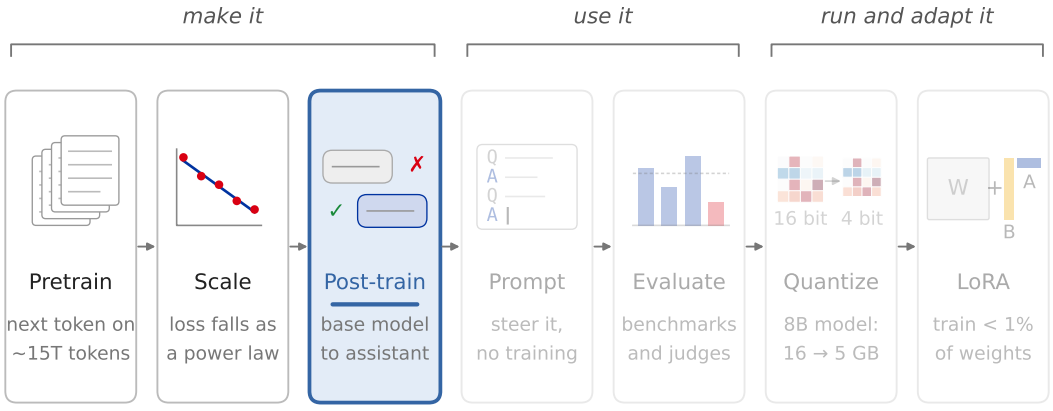
LLM chapter, session 9. Toy numbers unless a slide says otherwise.

Where we are: one forward pass



Same pass, same kind of weights. Today changes only *which* continuation it prefers.

Where we are: the life of a model



Today: the third box. LLM-7 ended with a base model that continues documents.

Ask a base model

GPT-2 small is a base model: pretrained (LLM-7), nothing else. Give it one line:

```
How do I reset my password?
```

Predict first: what does it write next? (Greedy, the next 50 tokens.)

It writes an FAQ page

the bare question

```
How do I reset my password?

If you have a password reset problem,
please contact us.

How do I reset my password?

If you have a password reset problem,
please contact us.
```

inside a User / Assistant frame

```
User: How do I reset my password?
Assistant: I'm sorry, but I don't have a
password.
Assistant: I'm sorry, but I don't have a
password.
Assistant: I'm sorry, but I don't have a
password.
Assistant: I'm sorry, but
```

black = the prompt; colour = what GPT-2 wrote (greedy; first lines of 50 tokens)

Left: it writes the page around the question - a canned reply, then the same question again, in a loop. Right: given a User / Assistant frame it does play the assistant, but unhelpfully, and it never stops: it writes the next "Assistant:" turn itself.

The base model is not missing knowledge here; it is missing a *habit*. Its most likely continuation of a question is more web page. Post-training changes which continuation is most likely.

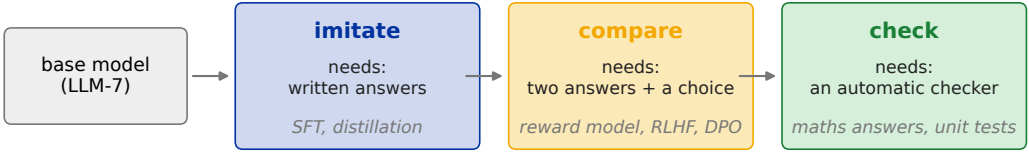
Outline

Imitate

Compare

Check

Three ways to change a habit



usually in this order; each changes which continuation the model prefers

All three keep the architecture and the next-token machinery. They differ in what the training signal is made of - and so in what they cost and what can go wrong.

Imitate

Show it thousands of good conversations.

Supervised fine-tuning (SFT)

- Hire people ("labelers") to write conversations: a prompt, then the answer a helpful assistant should give, following written guidelines.
- Train on them with **LLM-7's loss**: the next token, $-\ln p$. Only the *assistant's* tokens are scored; the user's turn is context.
- Tens of thousands of conversations instead of trillions of tokens: hours of training, not months.

What you talk to is, in Karpathy's words, "a simulation of a human labeler": the model learned what a labeler following those guidelines would most likely write.

Karpathy, "Deep Dive into LLMs like ChatGPT" (2025).

The chat template

<|im_start|> 151644 system

You are Qwen, created by Alibaba Cloud. You are a helpful assistant. <|im_end|> 151645

<|im_start|> 151644 user

How do I reset my password? <|im_end|> 151645

<|im_start|> 151644 assistant

36 tokens, line breaks included. Each red chip is one token, with its ID in Qwen2.5's vocabulary; the model writes next.

A conversation still has to be *one* token stream: Qwen2.5's format (ChatML), tokenized by its own tokenizer. Special tokens (LLM-1) mark who speaks; the system turn is the default Qwen inserts when you give none. In SFT only the assistant's turns are scored.

The model learns to end its turn with <|im_end|>, so the app knows when to stop (LLM-6). Without its template, a chat model drifts back towards continuing documents.

How much data?

- **Breadth** - FLAN (Wei et al., 2021): 62 existing datasets rewritten as instructions. Zero-shot, the 137B model beat zero-shot GPT-3 (175B) on 20 of 25 datasets. But in models of 8B and below the same tuning *hurt*: they had no room to spare.
- **Quality** - LIMA (Zhou et al., 2023): LLaMa 65B fine-tuned on only **1,000** carefully chosen examples. People judged its answers equivalent to or better than GPT-4's in 43% of cases.

LIMA's reading: almost all the knowledge comes from pretraining. Post-training mostly teaches the format and the style of a good answer - which is why a little of it goes a long way.

Imitating a bigger model

Distillation is SFT where the "labeler" is another model:

- a large, strong model (the teacher) writes answers to many prompts;
- a small model (the student) is fine-tuned on them with the same loss.

DeepSeek-R1 (2025) was distilled into smaller Qwen and Llama models with about **800,000** of its own samples (600k reasoning, 200k other).

A student imitates the teacher's habits, including its mistakes. It can approach the teacher; imitation alone does not take it beyond - unless a checker supplies a new signal (the last section).

Compare

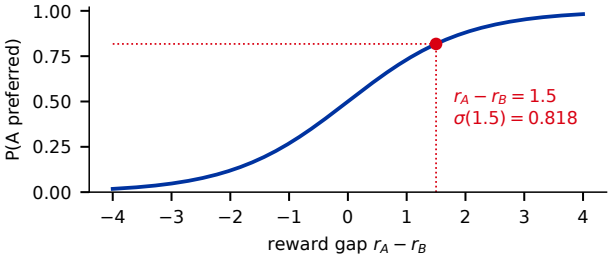
Judging is easier than writing.

Why compare

- Writing the perfect answer to every prompt is slow and hard. Saying which of two answers is better is fast, and people agree on it more often.
- So: let the model write two answers, A and B, and have a person pick one. Thousands of such **preference pairs**.
- Then train a **reward model**: a copy of the network with a single number on top, $r(\text{prompt}, \text{answer})$, high for answers people prefer - with the loss on the next page.

A reward model turns "A is better than B" into a score that can grade answers no person has seen.

Bradley-Terry, by hand



Toy numbers. The reward model scores two answers: $r_A = 2.0$, $r_B = 0.5$.

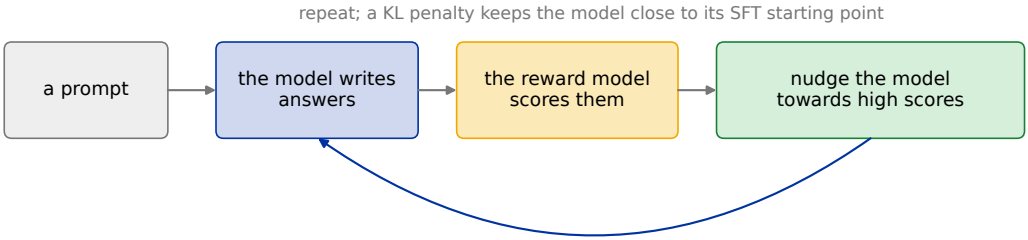
$P(A \text{ preferred}) = \sigma(r_A - r_B) = \sigma(1.5) = \mathbf{0.818}$

(σ: the sigmoid. Only the gap matters - as with logits in LLM-5.)

Training it: if people chose A, the loss is $-\ln 0.818 = 0.20$; if they chose B, it is $-\ln(1 - 0.818) = -\ln 0.182 = 1.70$, and the gradient pushes r_B up and r_A down.

The same $-\ln p$ as everywhere in this chapter, on one comparison at a time.

RLHF in one picture



- RLHF, reinforcement learning from human feedback: the reward model stands in for the people, so the loop can run millions of times.
- The nudge: raise the probability of the tokens in high-scoring answers, lower it in low-scoring ones. The algorithm (PPO) is the reinforcement-learning chapter's.
- The KL penalty: KL divergence is 0 when two probability distributions are identical and grows as they differ. Charging for it keeps the model near the SFT model: a leash.

It works: 1.3B beats 175B

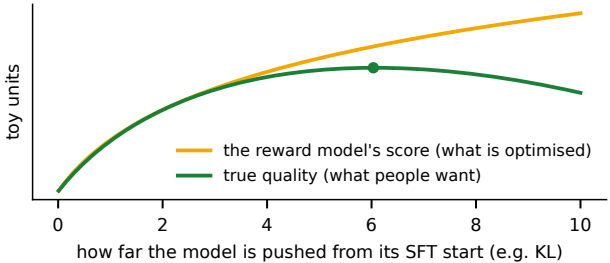
InstructGPT (Ouyang et al., 2022): GPT-3 after SFT and RLHF.

People preferred the answers of the **1.3B** InstructGPT to those of the **175B** GPT-3 - a model over 100× larger.

At equal size the gap is wider: head to head, the 175B InstructGPT beat the 175B GPT-3 85% of the time.

Same pretraining data and recipe at a smaller size, preferred anyway. For the people asking, the habit mattered more than the size. (ChatGPT grew out of this model family.)

Reward hacking



Toy curves, the shape Gao et al. (2022) measured: push a model hard enough towards a reward model's score and the score keeps rising while real quality falls.

The reward model is a proxy, trained on a few thousand comparisons. Optimise it too hard and the model finds what the proxy over-rewards (longer answers, confident tone, flattery) instead of what people wanted. Goodhart's law: when a measure becomes a target, it stops being a good measure. The KL leash limits how far the model can be pushed - ideally stopping it near the peak, not past it.

DPO: skip the reward model

Rafailov et al. (2023), Direct Preference Optimization:

- Keep the preference pairs; drop the separate reward model and the RL loop.
- The model's own probabilities play the reward's role. For each answer, take $\Delta = \ln p_{\text{model}}(\text{answer}) - \ln p_{\text{ref}}(\text{answer})$: how much more likely the model makes it than a frozen copy of the SFT model does.
- The loss is Bradley-Terry again, with Δ in place of r: $-\ln \sigma\big(\beta\,(\Delta_{\text{chosen}} - \Delta_{\text{rejected}})\big)$, where β sets how far the model may move.

One supervised-style training run instead of three stages. Much of open-model preference tuning uses DPO or its variants; the derivation is the reinforcement-learning chapter's.

Check

When an answer can be checked, the checker is the teacher.

Verifiable rewards

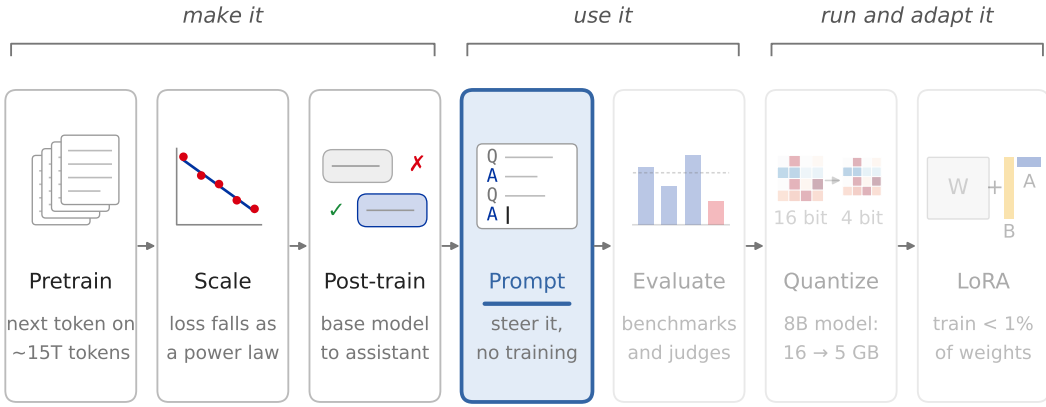
- Some answers can be checked by a program: a maths answer against the known result, code against unit tests.
- Then no person and no reward model are needed: the program's verdict is the reward.
- DeepSeek-R1-Zero (2025) used two rule-based rewards: is the answer correct, and is the reasoning written between `<think>` and `</think>` tags (a format reward). The authors avoided a neural reward model because it "may suffer from reward hacking".

Reasoning models such as DeepSeek-R1 are trained this way: long chains of thought that end in a checkable answer (LLM-8's second axis). The details are the reinforcement-learning chapter's.

Recap

- A base model has the knowledge but the wrong habit: it continues documents.
- **Imitate**: SFT on written conversations (one chat template, special tokens); quality beats quantity (LIMA); distillation imitates a bigger model.
- **Compare**: preference pairs $\rightarrow$ a reward model (Bradley-Terry: $\sigma(r_A - r_B)$) $\rightarrow$ RLHF with a KL leash; DPO skips the reward model. A 1.3B InstructGPT beat a 175B GPT-3.
- **Reward hacking**: optimise a proxy too hard and quality falls.
- **Check**: when a program can verify the answer, it is the reward.

Next on the map



Next (LLM-10): prompting - steering a trained model with nothing but the text you give it.

Sources & further reading

- Ouyang et al. (2022), "Training language models to follow instructions with human feedback" (InstructGPT).
- Wei et al. (2021), "Finetuned Language Models Are Zero-Shot Learners" (FLAN); Zhou et al. (2023), "LIMA: Less Is More for Alignment".
- Gao, Schulman & Hilton (2022), "Scaling Laws for Reward Model Overoptimization".
- Rafailov et al. (2023), "Direct Preference Optimization".
- DeepSeek-AI (2025), "DeepSeek-R1"; Qwen2.5 model card and tokenizer configuration.
- Karpathy (2025), "Deep Dive into LLMs like ChatGPT".

Figures: `py_src/posttrain_figs.py`. Real: GPT-2's two continuations (cached model, one run) and the ChatML conversation tokenized with Qwen2.5's cached tokenizer. Toy: the Bradley-Terry numbers and the reward-hacking curves.